

The semester is divided into five Epics, each spanning roughly 3 weeks. The Agile workflow will necessitate weekly sprints, with the team rotating the "Scrum Master" role to ensure all members gain leadership experience.

Team Roles:

- **Frontend Engineer (FE):** Responsible for the User Interface (UI), User Experience (UX), and client-side state management.
 - **Backend Engineer (BE):** Responsible for API development, database schema design, protocol implementation, and business logic.
 - **DevOps Engineer (DO):** Responsible for CI/CD pipelines, infrastructure management, security configuration, and monitoring.
 - **Testing Engineer (QA):** Responsible for unit tests, integration tests, end-to-end (E2E) testing, and validating protocol compliance.
-

Epic 1: Foundation – Identity, Actors, and Cryptography

Objective: The primary goal of Epic 1 is to establish the server infrastructure and implement the core Identity primitives. By the end of this phase, the system must be able to register users, generate decentralized identifiers, and expose an ActivityPub-compliant "Actor" profile that can be discovered by external Fediverse instances (e.g., Mastodon). This foundational work is critical as all subsequent social interactions rely on a valid, discoverable identity.

User Story 1.1: Decentralized Account Creation

Client Perspective: "As a new user, I want to sign up for an account that generates a secure, portable digital identity (DID) so that I can prove ownership of my username independently of the server administrators."

Context: This story sets the stage for data autonomy. Instead of just a database row, the registration process triggers the creation of a cryptographic key pair and a DID Document. The did:web method is chosen for its balance of decentralization and compatibility with web standards.¹⁵

- **Task 1.1.1 (BE):** Implement the did:web generation service. Upon registration, generate an RSA-2048 or Ed25519 key pair. Construct a W3C-compliant DID Document JSON containing the public key in the verificationMethod section.⁴
- **Task 1.1.2 (DO):** Configure the PostgreSQL database schema. Create a users table with columns for id (UUID), username, password_hash, and a jsonb column for the did_document. Ensure the private key is stored in a separate secrets table, encrypted at rest using pgcrypto or application-level encryption.¹³
- **Task 1.1.3 (FE):** Develop the registration form in React. Implement client-side validation for username availability and password strength. Upon successful submission, display the generated DID (e.g., did:web:instance.com:u:alice) to the user as their unique identifier.¹¹

- **Task 1.1.4 (QA):** Write unit tests for the key generation module. Verify that the generated DID Document passes the official W3C DID Core specification validator, ensuring correct syntax for id, controller, and assertionMethod.⁴
- **Task 1.1.5 (BE):** Implement the /.well-known/did.json resolution endpoint (or /.well-known/did/url-encoded-id). This endpoint must serve the DID Document with the content type application/did+ld+json or application/json, allowing external verifiers to fetch the public key.¹⁵

User Story 1.2: ActivityPub Actor Discovery (WebFinger)

Client Perspective: "As a user, I want my profile to be searchable by friends on other platforms (like Mastodon) using a familiar handle format (e.g., @alice@my-social-network.com)."

Context: ActivityPub relies on **WebFinger** (RFC 7033) to translate human-readable handles into machine-readable Actor URLs. Without this, the platform is an island, invisible to the rest of the Fediverse.¹⁹

- **Task 1.2.1 (BE):** Implement the /.well-known/webfinger endpoint using the Fedify framework's built-in WebFinger tools or a custom Node.js handler. It must accept a resource query parameter (e.g., acct:alice@domain.com).⁸
- **Task 1.2.2 (BE):** Configure the WebFinger response to return a JRD (JSON Resource Descriptor) object. This object must contain a subject (the acct URI) and a link with rel="self" pointing to the ActivityPub Actor URL (e.g., https://domain.com/users/alice).¹⁹
- **Task 1.2.3 (QA):** Create an integration test using a tool like curl or Postman to query the WebFinger endpoint. Assert that the response headers include Content-Type: application/jrd+json and that the body contains the correct links.²
- **Task 1.2.4 (DO):** Configure the web server (Nginx/Railway Router) to handle the /.well-known/ path prefix correctly, ensuring it is not blocked by static asset caching rules or security filters.¹⁷
- **Task 1.2.5 (FE):** Create a "Profile Settings" page where the user can copy their full handle (@user@domain) to share with friends. Implement a "Copy to Clipboard" feature for better UX.¹⁰

User Story 1.3: The Actor Profile (JSON-LD)

Client Perspective: "As a user, I want my public profile to contain my name, bio, and avatar in a format that other apps can understand, so they can display my identity correctly."

Context: The "Actor" object is the core identity primitive in ActivityPub. It is a JSON-LD document served at the user's ID URL. It defines the user's Inbox, Outbox, and Public Key.²²

- **Task 1.3.1 (BE):** Create the GET /users/:username endpoint. This endpoint must perform content negotiation: if the Accept header asks for application/activity+json (or ld+json), return the Actor JSON. If it asks for text/html, serve the React frontend.¹
- **Task 1.3.2 (BE):** Construct the Actor object structure using Fedify's type-safe builders. Include mandatory fields: id, type (Person), preferredUsername, inbox, outbox,

followers, following, and publicKey.²²

- **Task 1.3.3 (BE):** Integrate the DID Public Key into the Actor object. The `publicKey.publicKeyPem` field should match the key in the DID Document. This links the ActivityPub identity to the Decentralized Identity.⁵
- **Task 1.3.4 (FE):** Design the HTML fallback for the profile page. If a human visits the Actor URL in a browser, they should see a rendered React page with the user's avatar and bio, not raw JSON. This requires Server-Side Rendering (SSR) or proper routing logic.¹
- **Task 1.3.5 (QA):** Validate the generated Actor JSON against the official ActivityStreams validator or activitypub.academy test suite to ensure strict compliance with the spec (e.g., ensuring `@context` is an array containing the correct string).²⁵

User Story 1.4: Secure Authentication

Client Perspective: "As a registered user, I want to log in securely using my password so that I can access my account settings and post messages."

Context: While federation is decentralized, local access requires standard authentication. The project uses JWTs for stateless session management.²⁶

- **Task 1.4.1 (BE):** Implement a login endpoint that verifies the password against the stored bcrypt/argon2 hash. Upon success, issue a signed JWT containing the user's UUID and DID.²⁶
- **Task 1.4.2 (FE):** Build the Login UI. On successful login, store the JWT in a secure, HttpOnly cookie (if possible) or local storage, and update the global application state (Context API/Redux) to reflect the logged-in user.²⁷
- **Task 1.4.3 (DO):** Set up environment variables for `JWT_SECRET` and `COOKIE_SECRET` in the Railway/Render dashboard. Ensure these secrets are never committed to version control.²⁸
- **Task 1.4.4 (QA):** Perform security testing on the auth endpoints. Attempt SQL injection on the login form and verify that the API handles invalid credentials with generic error messages ("Invalid username or password") to prevent enumeration attacks.²
- **Task 1.4.5 (BE):** Create an authentication middleware that verifies the JWT on protected routes (e.g., `POST /outbox`). If the token is invalid or expired, return a 401 Unauthorized status.²⁶

User Story 1.5: Profile Customization

Client Perspective: "As a user, I want to upload a profile picture and write a short bio so that I can express my personality to the network."

Context: Updating the Actor object involves changing the database and ensuring the served JSON reflects these changes immediately.

- **Task 1.5.1 (FE):** Implement an image upload component using react-dropzone. The upload should send the file to a backend endpoint which streams it to an S3-compatible object storage (e.g., AWS S3, Cloudflare R2).²⁹

- **Task 1.5.2 (BE):** Create the PATCH /api/users/me endpoint. It accepts summary (bio), name (display name), and icon (avatar URL). Update the users table accordingly.¹³
- **Task 1.5.3 (BE):** Ensure the Actor JSON generation logic dynamically pulls these new fields. The icon property must be an Image object with a url and mediaType.¹⁹
- **Task 1.5.4 (QA):** Verify that the bio field sanitizes HTML input. Use a library like dompurify on the backend to prevent Stored XSS attacks if users attempt to inject <script> tags into their profiles.³⁰
- **Task 1.5.5 (DO):** Configure the object storage bucket policies to allow public read access for the uploaded avatar images, as they need to be fetchable by remote servers.²⁹

Epic 2: Content Creation and Data Autonomy

Objective: Epic 2 focuses on the "Outbox" mechanism. In ActivityPub, the Outbox is the user's publishing stream. This Epic covers the creation of posts (Notes), their storage in the database, privacy scoping, and the ability to edit or delete them. This establishes the "local" functionality of the social network before federation is introduced.

User Story 2.1: Publishing a Note

Client Perspective: "As a user, I want to write and publish a short text post (Note) so that I can share my thoughts with my followers."

Context: A "post" in ActivityPub is typically a Note object wrapped in a Create activity. The backend must construct this wrapper automatically.¹

- **Task 2.1.1 (FE):** Develop a "Composer" component. It should include a text area, a character counter (e.g., 500 limit), and a submit button. Implement "optimistic UI" updates where the post appears in the feed immediately upon submission.¹⁰
- **Task 2.1.2 (BE):** Create the POST /users/:username/outbox endpoint. This endpoint accepts the raw text content. The backend must generate a unique ID (URI) for the post, typically https://domain/users/alice/statuses/<uuid>.²
- **Task 2.1.3 (BE):** Construct the ActivityPub object. Create a Note object with content, published, attributedTo, and to/cc fields. Then, wrap this Note in a Create activity.³¹
- **Task 2.1.4 (DO):** Design the activities table in PostgreSQL. Use a jsonb column to store the full Activity object. This schema-less approach allows for flexibility in storing different Activity types (Notes, Articles, Images) without constant migrations.¹³
- **Task 2.1.5 (QA):** Write a test case that verifies the Create activity structure. Ensure the actor field matches the attributedTo field of the Note, and that the ids are distinct and valid URIs.²²

User Story 2.2: Viewing the Outbox (My Feed)

Client Perspective: "As a user, I want to see a chronological list of all the posts I have created so that I can review my own history."